

CyberNestSOAR

Technical Architecture & Engineering Report

Version	1.0
Date	May 2025
Classification	CONFIDENTIAL

► Table of Contents

Executive Summary (#exec)	Platform overview and key capabilities
1. System Architecture Overview (#s1)	Layered architecture, sensors, backend
2. Component Interaction & System Flow (#s2)	Pipeline, enrichment, risk scoring
3. Data Flow Between Services (#s3)	Telemetry pipelines and service channels
4. APIs and Integrations (#s4)	Endpoint reference and external services
5. Deployment Architecture (#s5)	Service inventory and infrastructure maturity
6. Constraints and Limitations (#s6)	Technical and operational constraints
7. Scalability Considerations (#s7)	Horizontal scaling, rate limits, production path
8. Operational Challenges (#s8)	Alert volume, enrichment reliability, playbooks
9. Security Considerations (#s9)	Network exposure, secrets, container hardening

10. Licensing and Compliance Analysis (#s10)	License overview and risk mitigation
11. Example Use Case: Phishing Attack (#s11)	End-to-end scenario walkthrough
Conclusion (#conclusion)	Architecture assessment and recommendations
Appendix (#appendix)	Environment variables reference

Executive Summary

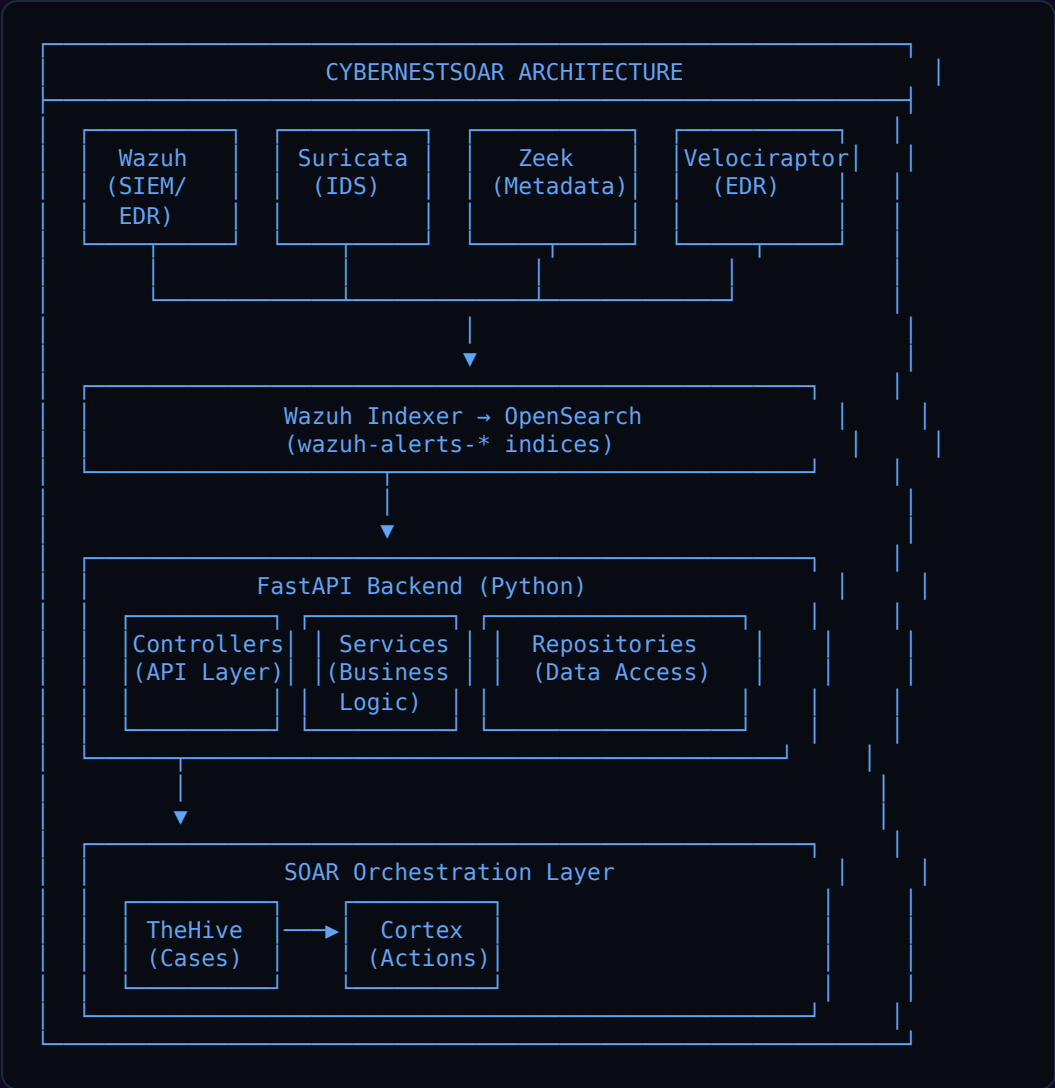
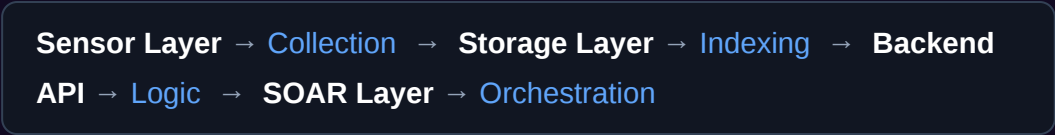
CyberNestSOAR is an **AI-driven** Security Orchestration, Automation and Response (SOAR) platform that unifies detection, enrichment, analysis, and automated response across a modern security operations environment. It integrates open-source tools into a coherent pipeline, orchestrated through a FastAPI backend and containerized via **Docker Compose**.

Telemetry Ingestion	Wazuh, Suricata, Zeek, Velociraptor
Alert Storage	OpenSearch (<code>wazuh-alerts-*</code>)
Threat Intelligence	VirusTotal, AbuseIPDB, MISP, EPSS, NVD, CISA KEV
Risk Scoring	Multi-factor 0–100 composite score
Automation	TheHive case management + Cortex response execution
Orchestration	FastAPI with <code>asyncio.gather()</code> parallel enrichment

The platform ingests security telemetry from network, host, and endpoint sensors, stores alerts in **OpenSearch**, and returns threat intelligence directly inside the `/api/v1/alerts/` response when `enrich=true`. The `enrichment_data` object includes VirusTotal, AbuseIPDB, MISP, EPSS, NVD, and CISA KEV where applicable, allowing the backend to drive automated playbooks through **TheHive** and **Cortex**.

1. System Architecture Overview

CyberNestSOAR is built around a layered modular architecture that separates concerns cleanly:



► 1.1 Architectural Layers

Layer	Components	Role
Sensor	Wazuh, Suricata, Zeek, Velociraptor	Alert and telemetry generation
Storage	OpenSearch, PostgreSQL	Alert indexing and relational data
Backend API	FastAPI (Python)	Orchestration, enrichment, risk scoring
SOAR	TheHive, Cortex	Case management, automated response

Intel	VT, AbuseIPDB, MISP, NVD, EPSS, KEV	Alert enrichment
Deployment	Docker Compose	Containerized service orchestration

► 1.2 Sensor Layer

The sensor layer is deliberately heterogeneous — each tool fills a distinct visibility gap:

Sensor	Type	Visibility	Output
Wazuh	SIEM / EDR	Host events, log analysis, FIM	Event → Indexer → OpenSearch
Suricata	NIDS	Signature-based intrusion detection	Alert → Filebeat → OpenSearch
Zeek	Net Metadata	Protocol-level (DNS, HTTP, SSL)	Log → Filebeat → OpenSearch
Velociraptor	EDR / Forensics	Deep endpoint inspection, live resp.	Query → Artifact → OpenSearch

All sensors funnel their output into **OpenSearch** through the **Wazuh** indexer, providing coverage across host, network, and endpoint dimensions.

► 1.3 Backend Architecture

The backend follows a standard layered pattern for a FastAPI application:

Layer	Module	Responsibility
Controllers	<code>classification.py</code> , <code>emails.py</code>	Handle API requests, route to services
Services	Enrichment, Scoring, Classify	Business logic and async orchestration
Repositories	OpenSearch, PostgreSQL	Abstract data access

Models	<code>UnifiedAlert</code> , <code>EnrichmentData</code> , <code>HostContext</code>	Unified data structures
---------------	--	-------------------------

i Note

Two controller files are active — `classification.py` and `emails.py` — reflecting the two primary outbound concerns: classifying alerts and sending notifications. The service layer carries most complexity around async enrichment and risk scoring.

2. Component Interaction & System Flow

CyberNestSOAR follows a pipeline model where each stage transforms or enriches data before passing it downstream.

► 2.1 End-to-End Pipeline

```

[Step 1]  Detection
  Wazuh / Suricata / Zeek / Velociraptor generates alert
  |
  ▼
[Step 2]  Storage – Alert written to OpenSearch via Wazuh indexer
  |
  ▼
[Step 3]  Retrieval – Backend API polls/receives alerts from OpenSearch
  |
  ▼
[Step 4]  Enrichment (inline)
  API returns alert with enrichment_data when enrich=true
  VT + AbuseIPDB + MISP + (EPSS + NVD if CVEs present)
  |
  ▼
[Step 5]  Classification – LLM-based → noise / important
  |
  ▼
[Step 6]  Risk Scoring – Composite score 0–100
  |
  ▼
[Step 7]  Playbook Decision – Score + tags → action + automation
  |
  ▼
[Step 8]  Response – TheHive creates case → Cortex executes action

```

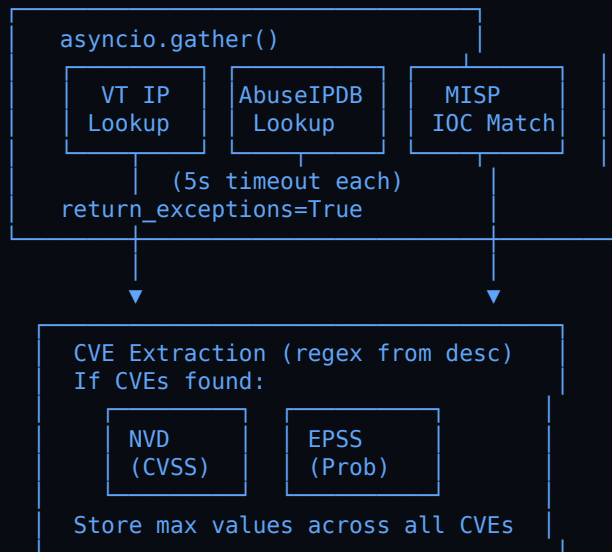
► 2.2 Enrichment Engine

Enrichment is embedded directly into the Alert endpoint. When `/api/v1/alerts/` is called with `enrich=true`, the API returns each alert with threat intelligence inside `enrichment_data`.

```

[Alert Retrieved from OpenSearch]
|

```



⚠ Private IP Handling

External lookups are skipped for private IPs; safe defaults applied (`vt_score=0` , `abuse_score=100`). This avoids wasting API quota on internal infrastructure while allowing CVE scoring.

Key design properties:

Property	Implementation
Concurrency	<code>asyncio.gather()</code> per-alert fan-out
Timeout	5 seconds per external service call
Fail-soft	<code>return_exceptions=True</code> — single failure never blocks batch
CVE handling	Regex extraction → max CVSS / EPSS across all CVEs

► 2.3 Risk Scoring

The risk scoring algorithm translates heterogeneous signals into a single **0–100** score:

```

RiskScore = min( (Severity × 10) + (CVSS × 5) + (EPSS × 50)
+ (AbuseScore × 0.5) + (VTScore × 0.5), 100 )

```

Component	Range	Max Contribution
Base score (severity × 10)	10–150	100 (clamped)

CVSS (× 5)	0–49	~49
EPSS (× 50)	0–50	~50
AbuseIPDB (× 0.5)	0–50	~50
VirusTotal (× 0.5)	0–50	~50

Priority mapping:

Score Range	Priority
> 70	CRITICAL
> 30	HIGH
≤ 30	MEDIUM LOW

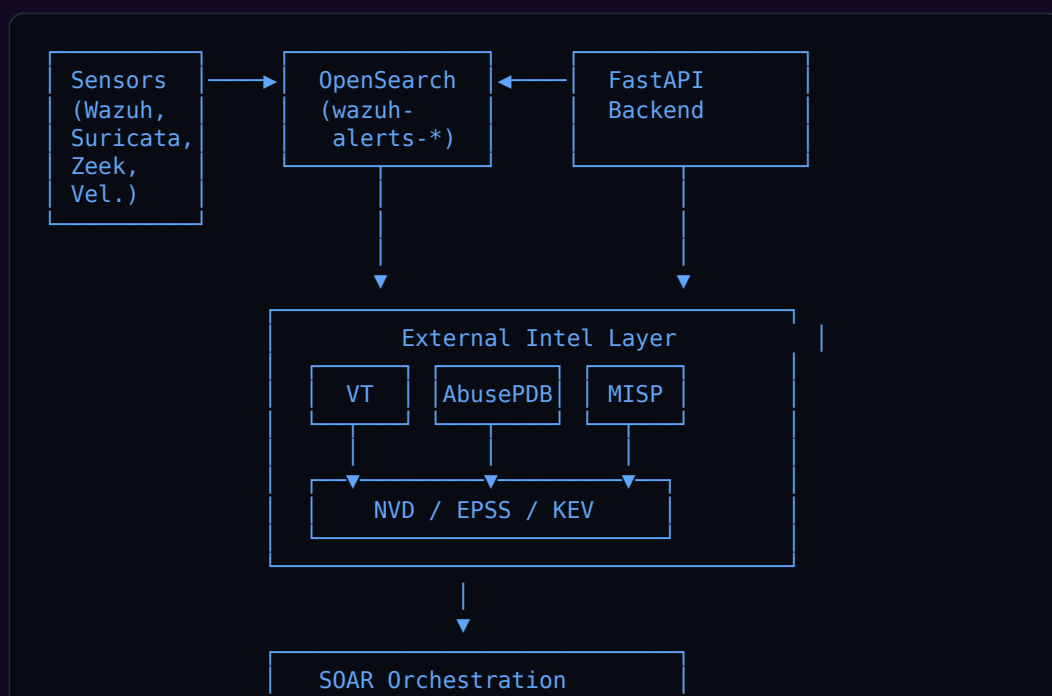
Note

Confidence is fixed at **0.85** across all scores, reflecting current pipeline maturity. Dynamic calculation based on enrichment completeness is recommended for future iterations.

3. Data Flow Between Services

Data flows through CyberNestSOAR in a largely unidirectional pipeline, with bidirectional communication at the SOAR and intelligence layers.

► 3.1 Primary Data Flows



TheHive → Cortex

Flow	Source → Destination	Proto	Notes
Telemetry	Sensors → OpenSearch	HTTP	All sensors write to <code>wazuh-alerts-*</code>
Alerts	OpenSearch → Backend API	REST	Pagination via <code>limit</code> / <code>offset</code>
IP Rep	Backend → VirusTotal	GET	4 req/min (public tier)
Abuse	Backend → AbuseIPDB	GET	Rate varies by subscription
IOC	Backend → MISP	GET	Matches set <code>malicious=true</code>
CVE	Backend → NVD	GET	6 req/sec (NIST policy)
EPSS	Backend → EPSS	GET	No rate limit, daily updated
Case	Backend → TheHive	REST	Enriched alert data
Response	TheHive → Cortex	REST	Block, quarantine, etc.
Sync	Backend → MISP	POST	Syncs last 50 alerts hourly

4. APIs and Integrations

The backend exposes a versioned REST API under `/api/v1/`. Each endpoint corresponds to one pipeline stage.

► 4.1 Core API Endpoints

Endpoint	Method	Function
----------	--------	----------

<code>/api/v1/alerts/</code>	GET	Retrieve alerts; <code>enrich=true</code> returns embedded intel
<code>/api/v1/risk-score/</code>	POST	Calculate risk score for a single alert
<code>/api/v1/patch/</code>	POST	Patch recommendations for CVEs
<code>/api/v1/alerts/filter</code>	POST	LLM-assisted noise vs important classification
<code>/api/v1/playbooks/decision</code>	POST	Automated action recommendation
<code>/api/v1/threat-intel/lookup</code>	POST	IOC lookup across VT, AbuseIPDB, MISP
<code>/api/v1/threat-intel/misp-sync</code>	POST	Sync recent alerts with MISP
<code>/health</code>	GET	Check OpenSearch and MISP connectivity

► 4.2 External Integrations Summary

Service	Purpose	Rate Limit	Auth
VirusTotal	IP/file reputation	4 req/min	<code>VT_API_KEY</code>
AbuseIPDB	IP abuse confidence	Varies	<code>ABUSEIPDB_KEY</code>
MISP	Internal IOC matching	Instance-dep.	<code>MISP_KEY</code>
NVD	CVE details / CVSS scores	6 req/sec	None
EPSS	Exploit probability	None	None
OpenSearch	Alert storage / query	N/A	Basic auth
TheHive	Case management	N/A	API key

Cortex	Automated response	N/A	Triggered by TheHive
CISA KEV	Known exploited vulns	N/A	In <code>enrichment_data</code>

5. Deployment Architecture

CyberNestSOAR is fully containerized using Docker Compose. All services run on a shared Docker network called `soc_net`, allowing direct service-to-service communication by container name.

⚠ Deployment Profile

This is a **lab-oriented deployment model** — suitable for development, testing, and small-scale production environments.

► 5.1 Service Inventory

Group	Services	Purpose
Backend	API, PostgreSQL, pgAdmin	Core logic, relational storage
SIEM / EDR	Wazuh Manager, Indexer, Dashboard, Agent	Host detection and alert indexing
Network Sensors	Suricata, Zeek	IDS and protocol metadata
Endpoint	Velociraptor	Deep inspection and live response
SOAR	TheHive, Cortex, Cassandra, ES, MinIO	Case mgmt, analysis, response
Threat Intel	MISP, MISP Modules, MySQL, Redis	IOC database and enrichment

► 5.2 Infrastructure Maturity

Area	Current	Target
------	---------	--------

Orchestration	Docker Compose	Kubernetes + Helm
Provisioning	Manual	Terraform
Configuration	Manual	Ansible
CI/CD	None (placeholder)	Pipeline defined
Scaling	Vertical only	Horizontal auto-scaling
HA	None	Multi-node clusters

⚠ Warning

Kubernetes, Terraform, CI/CD, and Ansible directories exist as placeholders only. The platform currently lacks automated provisioning, configuration drift detection, and orchestrated scaling.

6. Constraints and Limitations

⚠ Operational Honesty

Documenting system constraints is as important as describing capabilities. The following limitations apply to the current deployment.

► 6.1 Technical Constraints

Constraint	Impact	Severity
TheHive 5: commercial license required	Licensing cost barrier	CRITICAL
Velociraptor AGPLv3 — source disclosure	Legal exposure	HIGH
Risk confidence hardcoded at 0.85	Misleading on partial data	MEDIUM
MISP non-critical — enrichment continues	Silent detection degradation	MEDIUM
OpenSearch without SSL by default	Data in plaintext on network	CRITICAL
VT public API: 4 req/min	Bottleneck at scale	HIGH

⚠ Operational Caveat

An alert with only a base severity and no enrichment data receives the same confidence (0.85) as a fully enriched alert. This is misleading and should be addressed with dynamic confidence calculation tied to enrichment completeness.

► 6.2 Operational Constraints

Constraint	Risk
Docker Compose — no native HA	Full platform outage on host failure
No persistent volume management	Data loss on container restarts
No centralized backend logging	Errors captured only in <code>debug_info</code> fields

7. Scalability Considerations

► 7.1 Horizontal Scaling

Component	Score	Notes
FastAPI Backend	HIGH	Stateless — scales behind load balancer
Async I/O	HIGH	<code>asyncio.gather()</code> for I/O-bound enrichment
OpenSearch	HIGH	Horizontal via sharding + replication
PostgreSQL	LOW	Read replicas or distributed DB advised

► 7.2 External API Rate Limits

Service	Limit	Risk	Mitigation
VirusTotal	4/min	HIGH	Redis cache (TTL 24–48h)
NVD	6/sec	LOW	Local CVE data caching

EPSS	None	NONE	PostgreSQL with daily refresh
AbuseIPDB	Varies	MEDIUM	Caching + tier upgrade

💡 Recommendation

A **Redis-backed cache** for IP reputation lookups (TTL 24–48h) significantly reduces API calls for frequently seen IPs. EPSS scores cacheable locally in PostgreSQL with daily refresh.

► 7.3 Path to Production Scale

1. **Migrate** Docker Compose → Kubernetes for resilience and autoscaling
2. **Implement** Redis caching layer for external API responses
3. **Add** OpenSearch ILM policies to manage storage growth
4. **Deploy** API gateway for rate limiting and authentication
5. **Introduce** Kafka / message queue between sensors and backend

8. Operational Challenges

► 8.1 Alert Volume Management

SOAR platforms are prone to alert fatigue. The noise reduction endpoint (`/api/v1/alerts/filter`) uses **LLM-assisted classification**:

Feature	High Value Indicates
<code>event_count_5m</code>	Low → targeted attack; High → scanner noise
<code>unique_ips</code>	Low → single source; High → mass scanning
<code>vt_score</code>	High → malicious activity
<code>asset_criticality</code>	High + low noise indicators → important
<code>severity</code>	Low + high noise indicators → noise

⚠ Warning

LLM classifier quality directly determines utility. A poorly tuned model either suppresses real threats or floods analysts with noise.

► 8.2 Enrichment Reliability

External APIs can fail independently. The fail-soft architecture is graceful but creates risk:

```
[VT Unreachable]    →    risk_score understated
[AbuseIPDB Down]    →    risk_score understated
[MISP Unreachable] →    no IOC matching (silent)
```

⚠ Operational Guidance

Under degraded conditions, risk scores may be significantly understated — a **CRITICAL** alert might score **LOW** if VirusTotal is unreachable.

Monitor enrichment health proactively.

► 8.3 Playbook Maintenance

Playbook	Coverage	Maintenance
<code>detection_to_enrichment</code>	Alert → Enrich pipeline	Low (stable)
<code>enrichment_to_mediation</code>	Score → Action mapping	Medium (tune thresholds)
<code>phishing_response</code>	Phishing tag → IR workflow	High (evolving TTPs)

i Note

Decision quality is bounded by upstream enrichment and classification accuracy. Regular review cycles should be built into the operational model.

► 8.4 False Positive Management

Action	Risk	Mitigation
<code>QUARANTINE_HOST</code>	CRITICAL	Confirmation windows
<code>BLOCK_IP</code>	HIGH	Rollback capability
<code>CREATE_TICKET</code>	LOW	Non-disruptive

⚠ Warning

`QUARANTINE_HOST` and `BLOCK_IP` execute in **AUTO** mode for high-risk

alerts. Without a **human-in-the-loop checkpoint**, a misconfigured enrichment pipeline can cause unnecessary service disruption.

9. Security Considerations

First Principle

A security platform that introduces its own vulnerabilities creates an ironic problem.

► 9.1 Network Exposure

Issue	Risk	Recommendation
OpenSearch without SSL	CRITICAL	Enable TLS; mandatory outside isolated networks
No API authentication	CRITICAL	API key auth min; JWT + RBAC preferred
pgAdmin web exposed	HIGH	Never public; strong creds + MFA

► 9.2 API Key Management

Variable	Sensitivity	Storage Risk
VT_API_KEY	CRITICAL	Plain-text <code>.env</code> in version control
MISP_KEY	CRITICAL	Plain-text <code>.env</code> in version control
ABUSEIPDB_KEY	HIGH	Plain-text <code>.env</code> in version control
OS_USER / OS_PASS	HIGH	Plain-text <code>.env</code> in version control

Warning

Seven API keys stored in plain-text `.env` files. Use HashiCorp Vault, AWS Secrets Manager, or Kubernetes Secrets for production.

► 9.3 Container Security

Gap	Severity	Recommendation
No AppArmor / seccomp policies	HIGH	Document and apply hardening
Root user containers	MEDIUM	Non-root where possible
No network segmentation	MEDIUM	Restrict inter-service traffic

10. Licensing and Compliance Analysis

⚠ Legal Notice

CyberNestSOAR's use of open-source tools introduces real licensing obligations that must be understood before any commercial deployment.

► 10.1 License Overview

Component	License	Risk Assessment
Zeek	BSD 3-Clause	LOW — Permissive, no restrictions
Suricata	GPLv2	MEDIUM — Copyleft; safe in SaaS
Wazuh	GPLv2	MEDIUM — Same as Suricata
Velociraptor	AGPLv3	HIGH — Source disclosure risk
TheHive 5	Proprietary	CRITICAL — Commercial license required
Cortex	AGPLv3	HIGH — Same as Velociraptor
MISP	AGPLv3	HIGH — Internal deployment mitigates
FastAPI	MIT	LOW — Permissive, no restrictions

► 10.2 Risk Mitigation Strategy

The platform uses a **sidecar/API integration pattern** to reduce licensing risk. By accessing all tools through their REST APIs rather than embedding code,

derivative works under GPL/AGPL are avoided.

 Critical Action Item

TheHive 5 is the highest-risk item. Its proprietary license is **not** mitigated by API integration — production requires a commercial agreement with **StrangeBee**.

Note

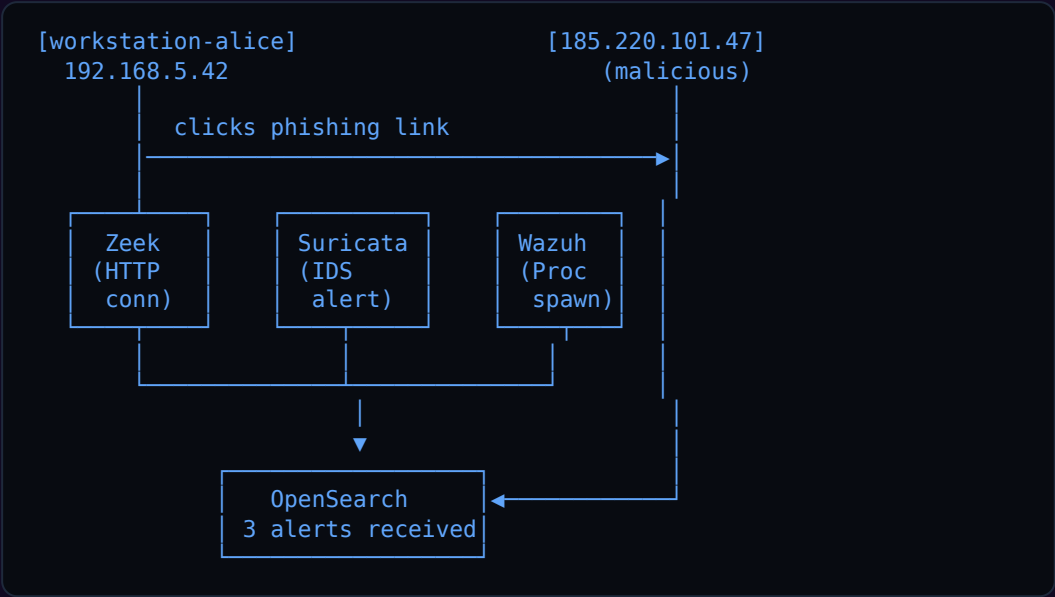
For Velociraptor/Cortex (AGPLv3): unmodified builds satisfy source disclosure by pointing upstream. Modifications must be made publicly available.

11. Example Use Case: Phishing Attack Scenario

The following scenario walks through how CyberNestSOAR handles a real phishing attack end-to-end.

Scenario: Credential Phishing via Malicious Email Link

• **Step 1 — Initial Detection**



User on **workstation-alice** (**192.168.5.42**) clicks a phishing link connecting to **185.220.101.47** (malicious). Three alerts land in OpenSearch:

Sensor	Detection	Details
Zeek	Outbound HTTP to malicious domain	Protocol-level log
Suricata	Phishing domain signature match	IDS alert

Wazuh	Suspicious process spawned	Host-based detection
-------	----------------------------	----------------------

• Step 2 — Alert Retrieval and Enrichment

Source	Result	Score
VirusTotal	Malicious detection rate: 78%	78
AbuseIPDB	Abuse confidence: 91	91
MISP	3 IOC matches (known phishing campaign)	malicious
CVE Extraction	No CVEs found	Skipped

Enriched Alert Summary

Source IP: 185.220.101.47
VT Score: 78 (malicious)
AbuseIPDB: 91 (high confidence)
MISP Matches: 3 events (phishing campaign)
Tags: ['suspicious', 'phishing', 'malicious-ip']
Severity: 8 (Wazuh rule level)

• Step 3 — Classification

Feature	Value	Signal
event_count_5m	4	Low → targeted attack
unique_ips	1	Low → single source
vt_score	78	High → malicious activity
asset_criticality	prod	High value target
Tags	phishing	Strong indicator

Result: Classified as **IMPORTANT** with **0.94 confidence**. Not filtered as noise.

• Step 4 — Risk Scoring

Base = $8 \times 10 = 80$ CVSS = $0 = 0$ EPSS = $0 = 0$
AbuseIPDB = $91 \times 0.5 = 45.5$ VirusTotal = 78×0.5
= 39 Enrichment = $45.5 + 39 = 84.5$ Final Score =
 $\min(80+84.5, 100) = 100$

Metric	Value
Risk Score	100 (capped)
Priority	CRITICAL
Confidence	0.85

• Step 5 — Playbook Decision and Response

With risk score **100** and **phishing** tag, the engine evaluates:

Condition	Trig.	Action	Auto
<code>risk >= 90</code>	✓	<code>QUARANTINE_HOST</code>	<code>AUTO</code>
Tag: <code>phishing</code>	✓	<code>CREATE_TICKET</code>	<code>SEMI</code>

Winner: Higher-priority rule → `AUTO` + `QUARANTINE_HOST`

Playbook Decision Output

Action: QUARANTINE_HOST
Automation Level: AUTO
Confidence: 0.95
Reason: Critical risk score (100) with confirmed malicious IP and phishing tag

Execution sequence:

```
[Playbook Decision]
├─ [TheHive] Creates case with 3 sensor alerts as evidence
├─ [Cortex] Executes quarantine on workstation-alice
└─ [SOC Alert] SEND_SOC_ALERT (severity >= 12)
    └─ SOC team notified of automated response
```

⚡ Performance Note

Full detection-to-quarantine pipeline completes in **under 30 seconds** — impossible with manual analyst workflows.

Conclusion

CyberNestSOAR represents a well-conceived SOAR platform that intelligently combines heterogeneous open-source security tools into a coherent automated workflow. The architecture makes sound engineering choices — async enrichment, fail-soft defaults, layered backend design — and the phishing scenario demonstrates that end-to-end automation is genuinely achievable at meaningful response speeds.

► Critical Path to Production

Priority	Area	Action Required
P0	Licensing	TheHive 5 commercial agreement; AGPLv3 review
P0	Security Hardening	OpenSearch TLS; API auth; secrets management
P1	Operational Tooling	Enrichment health monitoring; FP management
P2	Scalability	Kubernetes migration; Redis caching; ILM

✓ Final Assessment

Groundwork for scaling — Kubernetes placeholders, stateless API design, OpenSearch's horizontal scalability — is already laid. With focused investment in licensing, security hardening, and operational tooling, CyberNestSOAR has the architecture to serve as a **production-grade security operations backbone**.

Appendix: Environment Variables Reference

Variable	Description	Req.	Sensitivity
----------	-------------	------	-------------

<code>OS_HOST</code>	OpenSearch host URL	✓	MEDIUM
<code>OS_PORT</code>	OpenSearch port	✓	LOW
<code>OS_USER</code>	OpenSearch username	✓	HIGH
<code>OS_PASS</code>	OpenSearch password	✓	HIGH
<code>VT_API_KEY</code>	VirusTotal API key	✓	CRITICAL
<code>MISP_URL</code>	MISP instance URL	✓	MEDIUM
<code>MISP_KEY</code>	MISP API key	✓	CRITICAL
<code>ABUSEIPDB_KEY</code>	AbuseIPDB API key	Optional	HIGH

🔒 Security Notice

All secrets should be managed through a dedicated solution (HashiCorp Vault, AWS Secrets Manager, Kubernetes Secrets). Plain-text `.env` files in version control are unacceptable for production.